Deadline-Aware Food Delivery: Problem Template

Oct 24, 2026 · @Hanlin

Purpose

This is the one problem everyone in the group solves. Everyone uses the same data, the same parameters and the same scoring code, so all results can be compared.

The problem has two versions:

- **V1:** deliver as many orders on time as possible, with the lowest average wait.
- **V2:** same as V1, but no customer may wait more than 60 minutes.

Scenario

One rider picks up several orders at one restaurant and delivers all of them in one trip. Each dish stays fresh for a different time: ice cream lasts 30 minutes, rice lasts 90. The customer who ordered first is not always the one whose food spoils first.

Classic TSP assumption	Our use case
Every city is equally important	Each order has its own deadline
Only the visit order matters	The arrival time at each stop matters
One goal: shortest distance	Goals: most orders on time, then shortest average wait
Must visit every city	Orders that can't be on time are cancelled
The tour returns to the start	The route ends at the last customer

Problem definition

Input (all known at the start; no new orders arrive later)

- One restaurant r and n customers
- Travel time w(i, j) between any two places, in minutes
- For each customer k: order time o_k, food type, shelf life ℓ_k
- The rider leaves at t = 0, the time of the last order. No cooking time.

Output

- The order in which the rider visits customers, starting at r. Customers not in the route are cancelled.

Rules

- 1 rider, carries all orders, does not return to the restaurant
- Same travel time both ways, no traffic, no time spent at each door

- Wait of customer k = arrival time a_k − order time o_k
- Every delivered order must arrive by its deadline d_k:

Version	Deadline d_k	Meaning
V1	$o_k + \ell_k$	Food must not spoil
V2	$o_k + \min(\ell_k, 60)$	Food must not spoil, and no one waits more than 60 min

Goals (in this order)

1. Deliver as many orders as possible (cancel as few as possible)
2. Among those, lowest average wait of delivered customers

We always also report the longest wait and the number of cancelled orders.

Two useful facts about this problem:

- **Deadline = wait limit.** $a_k \le o_k + \ell_k$ is the same as "wait ≤ shelf life".
- **Order time does not change the goal.** Total wait = $\Sigma a_k - \Sigma o_k$, and Σo_k is fixed. So minimising average wait means minimising the sum of arrival times. Order time only matters through the deadline. This is known as the Minimum Latency Problem with deadlines, which is NP-hard.

Fixed parameters and data

These stay the same in every experiment. They are set in `common.py`; change them only if the whole group agrees.

Parameter	Value	Why
Restaurant	Bukit Bintang Plaza (GeoNameID 7284944, class S / MALL)	Busy food area in central KL
Customer locations	Populated places (class P) within 6 km of the restaurant: 33 places	Food delivery is within a city, so waits stay under about an hour
Travel time	Straight-line distance ÷ 20 km/h	Simple; the low speed covers detours, traffic lights and jams
Food types	Ice cream 30 min, noodles 60 min, rice 90 min; each order picks one at random	Different shelf lives create the conflict: who ordered first is not who is most urgent
Order times	Random within the 10 min before the rider leaves; the last order is at t = 0	All orders are known at the start
Wait cap (V2 only)	60 min	No customer waits more than an hour; this cuts only the rice deadline

Data used from MY.txt: columns GeoNameID, name, latitude, longitude and feature class. Duplicate coordinates are dropped.

Data we add: order time, food type and shelf life for each order. These are stored in the instance files, not in MY.txt, and are linked by GeoNameID. MY.txt itself is not edited.

Why these numbers: with 6 km, 20 km/h and these shelf lives, the best solution for 5 orders cancels 0.3 orders on average, and for 8 orders about 1.3 (12 random instances each). So the problem is hard but not hopeless.

Experiment variables

Only two things change between experiments. Run each setting on several seeds (for example 30) and report the mean and spread.

Variable	Flag	Suggested values	What it tests
Number of orders n	`--n`	5, 8, 10 (small); 20, 50, 100 (large)	Small: compare with the exact best. Large: speed, since brute force can't finish (10! ≈ 3.6 million orders, 15! ≈ 1.3 trillion)
Shelf-life tightness	`--tight`	0.5, 1.0, 1.5	Multiplies every shelf life; tighter means more conflict

When n is larger than 33, places are reused (several customers in the same neighbourhood). With one rider, large n will cancel many orders; those runs are mainly for speed.

Files and shared tools

All code is plain Python 3 with no extra libraries (fdr_template.zip).

File	What it does
`common.py`	The fixed parameters above, plus travel time and deadline helpers
`generate_instance.py`	Makes one instance from MY.txt: `--n`, `--seed`, `--tight`
`evaluate.py`	Scores a route for V1 or V2: delivered, cancelled, late, average wait, longest wait
`brute_force.py`	Exact best solution for n ≤ 10 (under 1 second)
`nearest_neighbor.py`	Baseline: classic nearest neighbour, then drops late orders

An instance file (JSON) holds the restaurant, the customers and the full travel-time matrix:

```
{"restaurant": {"geonameid": "7284944", "name": "Bukit Bintang Plaza", ...},
 "customers": [{"id": 1, "geonameid": "11054400", "name": "Golden Triangle",
                "order_time": -3.5, "food": "ice_cream", "shelf_life": 30.0}, ...],
 "travel_min": [[0, 2.63, ...], ...]}   # index 0 = restaurant, index k = customer k
```

How to plug in your own method: write `solve(inst, version)` that returns a list of customer ids in delivery order (ids left out are cancelled). Then score it with `evaluate(inst, route, version)`. A valid solution always has `late = 0`.

```
python generate_instance.py MY.txt --n 8 --seed 1
python brute_force.py instances/n8_s1_t1.0.json V1
python evaluate.py instances/n8_s1_t1.0.json V2 3 1 4 2
```

Worked example (n = 6, seed 10)

The best V1 route delivers all 6 orders, but Sentul waits 62.7 min. V2 moves Sentul earlier so everyone waits under 60 min, at a cost of 0.8 min on the average. Nearest neighbour cancels one ice cream.

File: `instances/example_n6_s10.json`. The rider leaves Bukit Bintang Plaza at t = 0.

Id	Customer	Ordered (min)	Food	Shelf life	From restaurant (min)
1	Golden Triangle	−3.5	Ice cream	30	2.6
2	Kampong Baharu	−3.2	Ice cream	30	6.9
3	Sentul	−5.3	Rice	90	14.8
4	Pudu	0.0	Noodles	60	5.3
5	Kuala Lumpur Village	−7.8	Ice cream	30	8.5
6	Taman Maluri	−3.2	Noodles	60	8.0

Solution	Route	Cancelled	Average wait (min)	Longest wait (min)
Best for V1	1 → 2 → 5 → 4 → 6 → 3	0	29.0	62.7 (Sentul)
Best for V2	1 → 2 → 5 → 3 → 4 → 6	0	29.8	55.2 (Taman Maluri)
Nearest neighbour (V1)	1 → 2 → 4 → 6 → 3	1 (Kuala Lumpur Village)	21.9	49.8

What this shows:

- **Nearest neighbour** goes to Pudu and Taman Maluri before the far ice cream at Kuala Lumpur Village, which then spoils. Its average wait looks lower only because it served one fewer customer. That is why the goals are compared in order: cancelled first, wait second.
- **V1** serves the three ice creams first and leaves the rice at Sentul for last, since rice lasts longest.
- **V2** cuts Sentul's deadline from 84.7 to 54.7, so the rider goes there fourth. The noodle customers wait longer instead.

Rules for the group

Every method must:

1. Use `generate_instance.py` for its inputs and `evaluate.py` for its scores. No private copies of the parameters.
2. Return only valid routes (`late = 0`).
3. Run on both V1 and V2.
4. Report for each setting: delivered, cancelled, average wait, longest wait, run time, and, for n ≤ 10, the gap to `brute_force.py`.
5. Compare results in goal order: fewer cancelled wins; average wait only breaks ties.

What counts as a new approach:

- **Counts:** a different algorithm idea (for example step-by-step greedy vs build a full route and then improve it) or a different main data structure.
- **Does not count:** the same code with different weights or parameters, or V1 vs V2. Those are experiments, not approaches.

Open questions for the group:

- ☐ Is 6 km / 20 km/h / 30-60-90 min OK for everyone?
- ☐ Which two approaches do we take forward as Approach A and Approach B?